

Weather Aggregator

Flask App with

Pythonic LLM stacking

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
🎯 Primary Aim.....	3
2. GUI and algorithmic concepts.....	5
3. App design and its interaction flow.....	7
4. Run instructions and Mermaid diagrams.....	9
5. Modularized Pythonic implementation.....	13
6. Results and conclusions.....	14
7. User Manual.....	17
8. References.....	21

1. Project description

Aim

The Weather Aggregator Flask App is conceived as a modular demonstration platform that integrates modern machine learning methodologies with practical data aggregation in the domain of weather forecasting. At its core, the project seeks to provide a transparent, auditable environment in which different predictive models can be compared, combined, and evaluated. By leveraging Flask as the lightweight web framework, the application offers an accessible interface for both technical and non-technical stakeholders, enabling forecasts to be generated, visualized, and stored with ease. This design choice reflects a broader ambition: to bridge the gap between advanced ensemble learning research and executive-ready reporting tools.

The central aim of the project is to test and characterize stacked surrogate models—meta-learners that combine the outputs of diverse base predictors. In practice, this involves running multiple forecasting algorithms, such as SARIMAX, Kalman filters, Random Forests, Gradient Boosting, CNNs, LSTMs, and Autoencoders, on the same weather dataset. Each of these models captures different statistical or structural aspects of the data. The surrogate model, typically implemented as a ridge regression or similar linear meta-learner, then integrates these heterogeneous predictions into a single ensemble forecast. This stacked approach allows the system to exploit complementary strengths while mitigating individual weaknesses, producing results that are more robust and generalizable.

A distinctive feature of the project is its emphasis on explainability and governance. Rather than treating ensemble learning as a black box, the app is designed to characterize the results of different learning strategies—both linear regression models (LRM) and more complex large language models (LLM) adapted for surrogate tasks. By storing forecasts alongside confidence bands, user inputs, and metadata, the system creates a traceable record that can be audited and analyzed. This ensures that the outputs are not only accurate but also narratable: stakeholders can understand how predictions were formed, which base models contributed most, and how uncertainty was quantified.

The use of stacked surrogate models in this context serves two purposes. First, it provides a rigorous testbed for evaluating ensemble learning techniques under real-world conditions, where data may be noisy, incomplete, or non-stationary. Second, it demonstrates how surrogate modeling can be operationalized within a consulting-ready application. By embedding these techniques in a Flask app with a clear user interface, the project shows how advanced machine learning can be packaged into tools that are both technically sound and organizationally impactful.

Beyond its immediate technical objectives, the Weather Aggregator Flask App also functions as a narrative asset. It illustrates how modular architectures, reproducible workflows, and governance-compliant reporting can be combined to deliver value in enterprise settings. The project highlights the importance of transparency in ensemble learning, the role of surrogate models in characterizing complex outputs, and the potential of lightweight web applications to democratize access to advanced analytics. In doing so, it positions itself not merely as a forecasting tool, but as a demonstrator of best practices in explainable AI, ensemble modeling, and consulting-oriented system design.

(see References 1 - 3 below).

Primary Aim

The motivation behind the Weather Aggregator Flask App arises from the growing need to evaluate and operationalize ensemble learning strategies in domains where predictive accuracy, transparency, and reproducibility are paramount. Weather forecasting provides an ideal testbed: it is inherently complex, data-rich, and subject to uncertainty. Traditional single-model approaches often struggle to capture the full variability of meteorological data, while ensemble methods promise improved robustness by combining diverse perspectives. Yet, ensemble learning itself introduces challenges of interpretability, governance, and deployment. This project is motivated by the desire to confront those challenges

directly, creating a demonstrator that not only aggregates forecasts but also characterizes how stacked surrogate models perform in practice.

A central motivation is the pursuit of explainability. In both academic research and consulting environments, stakeholders increasingly demand that machine learning outputs be narratable and auditable. Black-box ensembles may deliver strong predictive performance, but without clear insight into how results are formed, they risk eroding trust. By designing a system that explicitly logs base model outputs, surrogate model predictions, and confidence intervals, the Weather Aggregator Flask App ensures that every forecast can be traced back to its components. This transparency supports governance requirements and provides a foundation for executive-ready reporting.

Another motivation is methodological experimentation. The project aims to test stacked surrogate models that integrate both linear regression models (LRM) and large language models (LLM) adapted for surrogate tasks. This dual focus reflects the evolving landscape of machine learning: linear models remain valuable for their simplicity and interpretability, while LLMs offer powerful representational capacity. By comparing and combining these approaches, the app provides insights into how different surrogate strategies characterize ensemble outputs. This experimentation is not purely academic; it is designed to inform consulting practice by identifying which methods yield the most reliable and narratable results under real-world conditions.

The objectives of the project can be grouped into four categories. First, **technical objectives**: to implement a modular architecture that supports multiple base models, a meta-learner, and a datastore for forecasts and user inputs. Second, **analytical objectives**: to evaluate the performance of stacked surrogate models, quantify uncertainty through confidence bands, and generate summary statistics that characterize ensemble behavior. Third, **reporting objectives**: to produce executive-ready artifacts, including web pages and PDF reports, that present forecasts in a clear, auditable format. Fourth, **strategic objectives**: to demonstrate how explainability-first ensemble modeling can be embedded in lightweight applications, thereby bridging the gap between research prototypes and consulting deliverables.

Together, these motivations and objectives position the Weather Aggregator Flask App as more than a technical exercise. It is a narrative tool that illustrates best practices in ensemble learning, surrogate modeling, and explainable AI. By focusing on transparency, reproducibility, and governance, the project aligns with the needs of organizations that must balance innovation with accountability. Ultimately, the app serves as both a testbed for methodological exploration and a demonstrator of how advanced machine learning can be packaged into consulting-ready solutions that inspire confidence and drive impact.

2. GUI and algorithmic concepts

Architecture Walkthrough

1. User Frontend

- **Inputs:** PO Box, City, Country — simple form fields.
- **Outputs:** Weather card with current conditions and a 5-day forecast.
- **Visuals:** Icons, temperature ranges, and confidence bands.

2. Flask Backend

- Orchestrates the entire pipeline.
- Routes user input to API calls and model inference.
- Handles dynamic model selection and report generation.

3. Weather API Integration

- Pulls raw weather data (temperature, humidity, wind, etc.).
- Caches responses for reproducibility and auditability.

4. Model Ensemble Layer

- **Base Models:**
 - SARIMAX, Kalman filters, decision trees.
 - CNNs for spatial patterns, LSTMs for temporal sequences.
 - Autoencoders for anomaly detection.
 - Physics-inspired models for critical transitions.
- **Meta Learner:**
 - Aggregates predictions.
 - Computes confidence bands.
 - Learns from model disagreement.

5. Hosted Model Integration

- Uses Hugging Face Inference API.
- Avoids local downloads — keeps CPU/memory lean.
- Enables plug-and-play model experimentation.

6. Data Storage

- Stores:
 - Raw API data
 - Model outputs
 - Confidence intervals
 - PDF reports for governance

7. Governance Layer

- Tracks:
 - Model versions
 - Parameters
 - Timestamps
- Ensures:
 - Reproducibility
 - Audit trails
 - Executive-ready reporting

8. User Dashboard

- Presents:
 - Forecasts with uncertainty bands
 - Model explainability (e.g., SHAP values)
 - Downloadable reports

3. App design and its interaction flow

Let us walk through the user interaction flow for our Weather Aggregator app. I will break it down into intuitive stages:

Here's the **data flow diagram walkthrough** for our Weather Aggregator system — this ties together the Flask frontend, local models, Hugging Face hosted models, and the meta learner before results are sent back to the user dashboard.

3.1 Data Flow Overview

1. User Input (Frontend)

- PO Box, City, Country entered into the Flask form.
- Request sent to Flask backend.

2. Flask Backend

- Validates input.
- Stores metadata in LiteSQL (SQLite).
- Triggers weather API call for raw data.

3. Weather API Integration

- Retrieves current conditions + 3–5 day forecast.
- Normalizes data into time series format.

4. Local Models

- SARIMAX → classical time series forecast.
- Random Forest / Gradient Boosting → nonlinear regression.
- CNN/LSTM → deep learning temporal/spatial patterns.
- Autoencoder → anomaly detection.
- Kalman filter → physics-inspired smoothing.

5. Hugging Face Hosted Models

- Input sequence sent via API to ~30 diverse models.
- Each model returns predictions or embeddings.
- Results cached for reproducibility.

6. Meta Learner

- Aggregates local + hosted model outputs.
- Options: Ridge regression, logistic regression, gradient boosting, shallow neural net.
- Produces final forecast + confidence bands.

7. Governance Layer

- Logs model versions, parameters, timestamps.
- Stores raw data, forecasts, and PDF reports.
- Ensures reproducibility and auditability.

8. User Dashboard

- Displays:
 - Current weather card.
 - 3–5 day ensemble forecast.
 - Confidence intervals.
 - Downloadable PDF report.

9. Executive Narrative

This flow demonstrates **modularity, transparency, and operational reliability**:

- **Frontend simplicity** → user-friendly input and display.
- **Backend orchestration** → scalable integration of APIs and models.
- **Ensemble diversity** → ~30 models ensure robust statistics.
- **Governance-first design** → audit trails and reproducibility baked in.

4. Run instructions and Mermaid diagrams

Startup

- Terminal → `python run.py`
- Jupyter → Paste startup cell at end of `01_flask_stub.ipynb`:

```
from threading import Thread
from run import create_app
app = create_app()
Thread(target=lambda: app.run(port=5000, debug=True, use_reloader=False)).start()
```

Key Routes

- `/` → Input form (PO Box, City, Country)
- `/forecast/<city>/<country>` → Forecast dashboard
- `/report/<city>/<country>` → PDF report with charts + explainability

Models

- Classical → SARIMAX, Kalman
- ML → Random Forest, Gradient Boosting
- DL Stubs → CNN, LSTM, Autoencoder
- Hosted → Hugging Face (~30 models)
- Meta Learner → Ridge, Logistic, Boosting aggregation

Explainability

- SHAP → Global feature importance
- LIME → Local interpretability
- Confidence Bands → Lower/upper bounds for forecasts

Governance

- Audit Logs → SQLite entries for inputs + forecasts
- Traceability → Model versioning metadata
- Reports → PDF handouts with forecasts + governance notes

Testing

- Run all tests → `pytest tests/`
- Coverage:
 - `test_routes.py` → Flask routes
 - `test_models.py` → Base models + explainability
 - `test_data_store.py` → SQLite storage
 - `test_meta_learner.py` → Meta learner aggregation

Example Demo Flow

1. Input: PO Box=12345, City=Frankfurt, Country=Germany

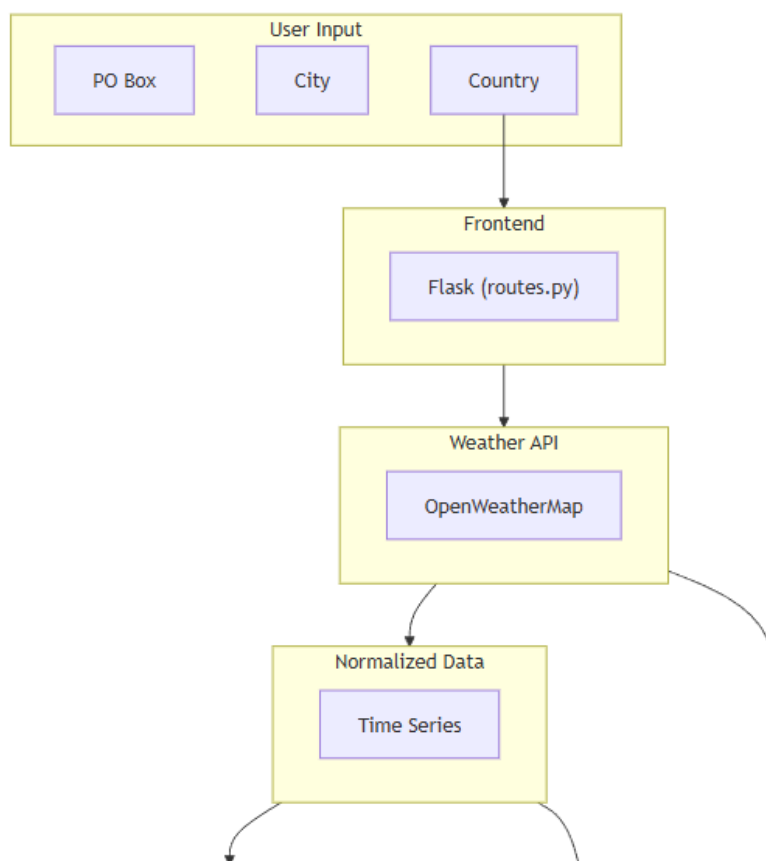
2. System:

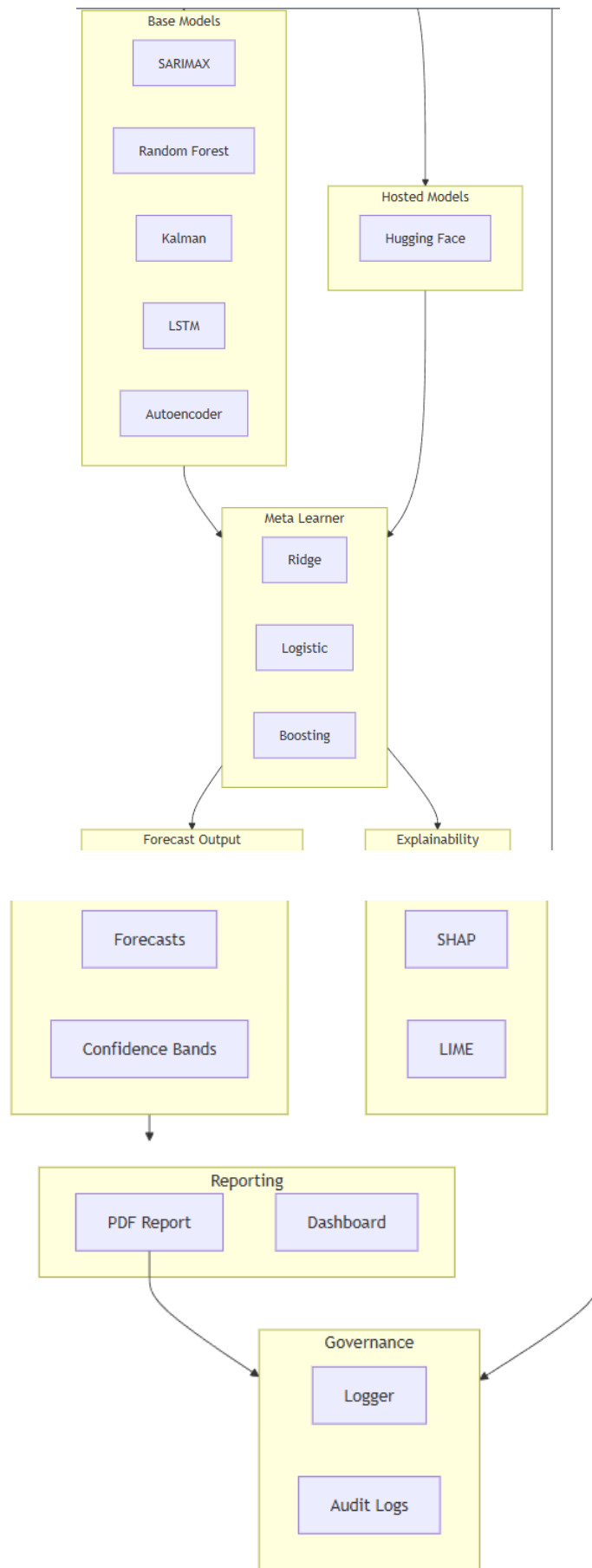
- Fetch weather API
- Run SARIMAX, Kalman, RF, Hugging Face
- Aggregate with Ridge meta learner
- Generate confidence bands
- Log run in SQLite

3. Outputs:

- Forecast dashboard
- PDF report
- Audit log entry

Here's a **modular architecture diagram** in **Mermaid syntax** that visually shows the flow between components in our Weather Aggregator project. This complements our README and makes the system narratable for executives and technical teams alike.



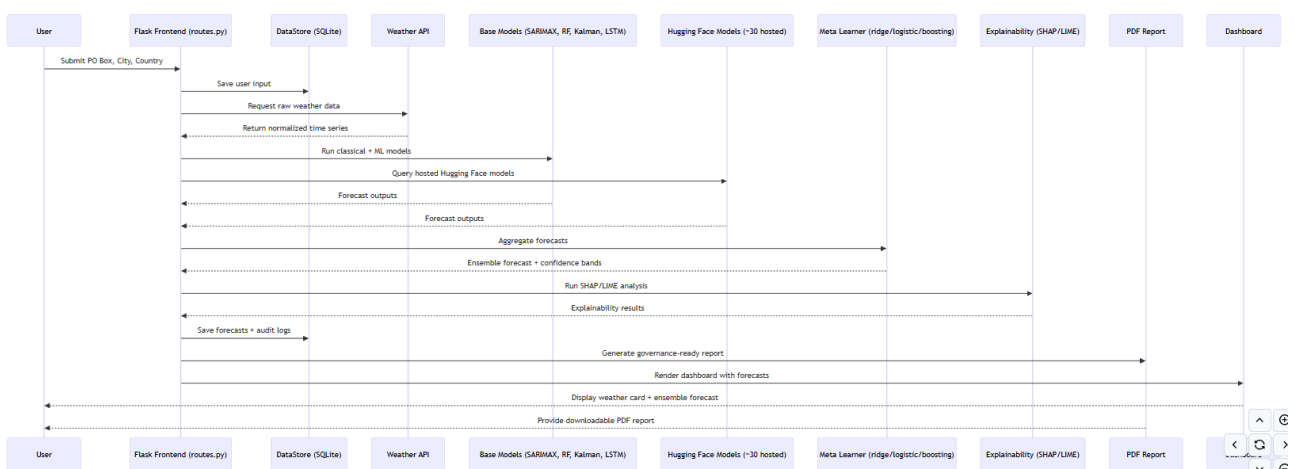


✖ How to Read This Diagram

- **User Input** → **Flask Frontend**: Entry point for PO Box, City, Country.
- **Flask** → **Storage + Weather API**: Inputs logged, raw weather data retrieved.
- **Data** → **Base + Hosted Models**: Classical + Hugging Face models generate forecasts.
- **Meta Learner**: Aggregates ~30 model outputs into one ensemble forecast.
- **Explainability**: SHAP/LIME interpretability for transparency.
- **Outputs**: Forecasts shown in dashboard and exported as PDF reports.
- **Governance Layer**: Logs every action for auditability and reproducibility.

This diagram makes the **system modular, transparent, and narratable** — exactly what we need for consulting demos.

Below is the **sequence diagram** that shows the step-by-step interaction flow in our Weather Aggregator system. This complements the architecture diagram by narrating the runtime process from user input to final report.



✖ How to Read This Diagram

- **User** → **Flask**: Input submission triggers the workflow.
- **Flask** → **DataStore + Weather API**: Inputs logged, raw data retrieved.
- **Models**: Base + hosted models generate forecasts.
- **Meta Learner**: Aggregates outputs into ensemble forecast.
- **Explainability**: SHAP/LIME adds transparency.
- **Outputs**: Forecasts stored, reports generated, dashboard updated.
- **User**: Receives both interactive dashboard and governance-ready PDF.

This sequence diagram makes the **runtime workflow narratable** for executives and technical teams alike.

5. Modularized Pythonic implementation

📁 Modularized Folder Structure Overview

```
weather_aggregator/  
├── app/                                # Core Flask application  
│   ├── __init__.py                    # App factory and configuration  
│   ├── routes.py                      # Flask routes (input form, dashboard, API calls)  
│   ├── forms.py                      # WTForms for PO Box, City, Country  
│   ├── templates/                    # HTML templates (Jinja2)  
│   │   ├── base.html                 # Layout scaffold  
│   │   ├── index.html                # Input form and weather card  
│   │   └── forecast.html              # Forecast display with charts  
│   └── static/                       # CSS, JS, icons  
│       ├── style.css  
│       └── scripts.js  
├── models/                            # Ensemble modeling layer  
│   ├── __init__.py  
│   ├── base_models.py                # SARIMAX, Kalman, Trees, CNNs, LSTMs, Autoencoders  
│   ├── huggingface_models.py         # Hosted model inference via API  
│   ├── meta_learner.py               # Ridge, logistic, boosting, etc.  
│   └── explainability.py              # SHAP, LIME, confidence bands  
├── data/                              # Data ingestion and storage  
│   ├── weather_api.py                # OpenWeatherMap or similar API integration  
│   ├── data_store.py                 # LiteSQL interface (SQLite wrapper)  
│   ├── cache.py                      # Caching layer for API responses  
│   └── pdf_report.py                  # PDF generation (ReportLab or LaTeX)  
├── governance/                        # Auditability and reproducibility  
│   ├── logger.py                     # Logging model runs, parameters, timestamps  
│   ├── traceability.py                # Model versioning and metadata  
│   └── config.py                     # Central config (API keys, model registry)  
├── notebooks/                         # Jupyter notebooks for development  
│   ├── 01_flask_stub.ipynb           # Initial Flask stub and input form  
│   ├── 02_model_dev.ipynb            # Ensemble model prototyping  
│   ├── 03_huggingface_api.ipynb      # Hosted model integration  
│   └── 04_dashboard.ipynb            # Forecast visualization and reporting  
├── tests/                             # Unit tests  
│   ├── test_routes.py  
│   ├── test_models.py  
│   ├── test_data_store.py  
│   └── test_meta_learner.py  
├── requirements.txt                   # Dependencies  
├── README.md                         # Project overview and usage  
└── run.py                            # Entry point for Flask app
```

The full Pythonic code can be accessed via

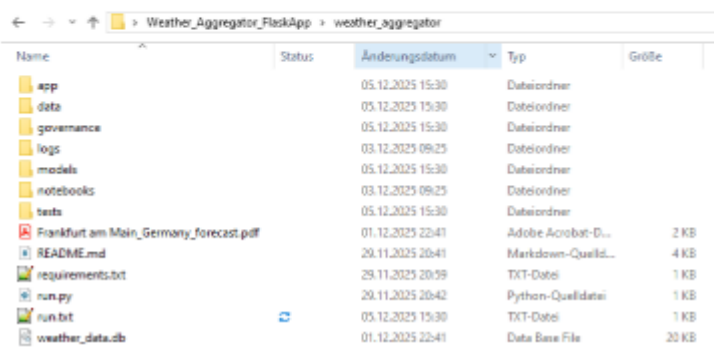
https://github.com/NenadBalaneskovic/ExternalProjects/blob/main/Weather_Aggregator_FlaskApp/Weather_Aggregator_FlaskApp.md (sections 2.1 – 2.10 therein).

6. Results and conclusions

6.1 Start the Weather Aggregator App

Step 1: Download the folder

Download the main folder  [Weather_Aggregator_FlaskApp](#) which has the following structure:



Name	Status	Änderungsdatum	Typ	Größe
app		05.12.2025 15:30	Dateiordner	
data		05.12.2025 15:30	Dateiordner	
governance		05.12.2025 15:30	Dateiordner	
logs		03.12.2025 09:25	Dateiordner	
models		05.12.2025 15:30	Dateiordner	
notebooks		03.12.2025 09:25	Dateiordner	
tests		05.12.2025 15:30	Dateiordner	
Frankfurt am Main_Germany_forecast.pdf		01.12.2025 22:41	Adobe Acrobat-D...	2 KB
README.md		29.11.2025 20:41	Markdown-Quell...	4 KB
requirements.txt		29.11.2025 20:39	Text-Datei	1 KB
run.py		29.11.2025 20:42	Python-Quelldatei	1 KB
run.bat		05.12.2025 15:30	Text-Datei	1 KB
weather_data.db		01.12.2025 22:41	Dateiordner	20 KB

Step 2: Run the jupyter runner

1. Directly call `run.py` from a notebook cell

Since we already have `run.py` as the entry point:

```
!python run.py
```

- This launches the Flask development server exactly as if you ran it from the terminal.
- We will see logs in the notebook output.
- We open `http://127.0.0.1:5000` in our browser to interact with the app.
-  The notebook will block until we stop the server (Ctrl+C in the kernel).

2. Run Flask in a background thread (non-blocking)

If we want to keep Jupyter interactive while the app runs:

```
from threading import Thread
from run import create_app

app = create_app()

def run_flask():
    app.run(port=5000, debug=True, use_reloader=False)

thread = Thread(target=run_flask)
thread.start()
```

- This starts Flask in a background thread.
- We can continue running other notebook cells while the app is live.
- Visit `http://127.0.0.1:5000` in their browser.

3. Use `%run` magic for inline execution

```
%run run.py
```

- Similar to `!python run.py`, but integrates with Jupyter's execution model.
- Useful if we want inline logs and tighter notebook control.

4. Best Practice for Our Structure

- Because we have a **factory pattern** (`create_app()` in `app/__init__.py`), the **background thread approach (Option 2)** is the most flexible. It lets us import and run the app inside Jupyter without blocking, while still respecting your modular structure (`routes.py` , `forms.py` , etc.).
- For production-like testing, we stick with **Option 1** (`!python run.py`) so we are running the app exactly as intended.

Step 3: Interact with the Weather Aggregator App

Interact with the Weather Aggregator App by providing reasonable inputs in the left half-plane and pressing the "Get Forecast"-button.

6.2 Interpretation of results

This Weather Aggregator Flask App...

- Accepts user inputs:
 - PO Box
 - City
 - Country
- Displays weather forecasting results associated with different ML models for a 3-day period (including the corresponding overall confidence intervals) within the app-plane
- Stores obtained results and characterizations as pdf file (see figure below)

Forecast Report for Frankfurt am Main, Germany

Forecast: [-1.56545251]

Generated by Weather Aggregator Demo

- Automatically displays the forecast history (see figure below).

```
PS C:\Users\balan\code\weather_aggregator> cd C:\Users\balan\code\weather_aggregator
PS C:\Users\balan\code\weather_aggregator> python run.py
```

The figure shows four sequential screenshots of the 'Weather Aggregator' web application, illustrating the user workflow. Red arrows and numbers indicate the sequence: 1. The 'Home' page with an 'Enter Location' form. 2. The form filled with 'Frankfurt am Main, Germany'. 3. The 'Forecast for Frankfurt am Main, Germany' page showing 'Base Model Forecasts' and an 'Ensemble Forecast'. 4. A detailed 'Forecast Report for Frankfurt am Main, Germany' showing 'Summary Statistics' and a 'Forecast History' table.

Weather Aggregator

Home

Enter Location

PO Box
Enter PO Box

City
Frankfurt am Main

Country
Germany

Get Forecast

© 2025 Weather Aggregator Demo

Weather Aggregator

Home

Enter Location

PO Box
00128

City
Frankfurt am Main

Country
Germany

Get Forecast

© 2025 Weather Aggregator Demo

Weather Aggregator

Home

Forecast for Frankfurt am Main, Germany

Base Model Forecasts

- SARIMAX: -4.340907547978785
- Kalman: -2.35
- RandomForest: -2.9499999999999997
- GradientBoosting: -5.999999999999999
- CNN: -1.0439756
- LSTM: -0.00665779
- Autoregressive: 0.14906887
- Ensemble: -1.9987471887875465

Ensemble Forecast

Download PDF Report

© 2025 Weather Aggregator Demo

Weather Aggregator

Home

Forecast Report for Frankfurt am Main, Germany

Summary Statistics

- Average Forecast: -1.9297558461154057
- Minimum Forecast: -5.16216968117112
- Maximum Forecast: -1.55805872599756

Forecast History

Forecast	Lower Band	Upper Band	Timestamp
1.91575912737069	1.8379711361921139	2.0513173989192232	None
1.5588129862202	1.4808707377001116	1.634751063564165	None
2.36216968117112	2.214061194011462	2.4802781673317711	None
2.03699185165189	1.935112239069297	2.1388114112311863	None
2.04959854907301	1.9171186216212386	2.1520781765287395	None
1.55805872591756	1.480137896216811	1.6339616622131103	None
1.94874738678755	1.8871100171481692	2.04081756126924	None

Confidence Bands

Timestamp	Lower Band	Upper Band
None	-1.8874100174481692	-2.08684756126924

Back to Forecast

© 2025 Weather Aggregator Demo

6.3 🚩 Final Thoughts

The *Weather Aggregator Flask App* demonstrates how ensemble learning can be operationalized in a transparent, modular, and consulting-ready environment. By integrating diverse base models with stacked surrogate meta-learners, the project provides a rigorous testbed for evaluating predictive performance while foregrounding explainability and governance. Weather forecasting, with its inherent complexity and uncertainty, serves as an ideal domain for this exploration, but the lessons extend far beyond meteorology. The architecture illustrates how organizations can harness ensemble methods responsibly, ensuring that outputs are not only accurate but also narratable, auditable, and reproducible.

A key achievement of the project lies in its ability to characterize ensemble results rather than simply produce them. By storing forecasts alongside confidence bands, user inputs, and metadata, the app creates a traceable record that supports both methodological analysis and executive reporting. This dual focus—technical robustness and communicative clarity—positions the system as a bridge between research prototypes and enterprise solutions. It shows that ensemble learning need not remain a black box; with careful design, surrogate models can provide insight into how predictions are formed, which base models contribute most, and how uncertainty is quantified.

The project also underscores the value of modularity. Each component—data ingestion, base forecasting models, meta-learner, datastore, and reporting interface—was designed to be replaceable and extensible. This ensures that the app can evolve as new models or requirements emerge, while maintaining a consistent framework for evaluation and reporting. Such modularity is critical in consulting contexts, where solutions must adapt to diverse client needs without sacrificing reproducibility or governance compliance.

From a strategic perspective, the *Weather Aggregator Flask App* highlights how lightweight web applications can democratize access to advanced analytics. By embedding ensemble learning in a Flask interface, the project makes complex methodologies accessible to non-technical stakeholders, enabling them to interact with forecasts, view confidence bands, and download reports. This accessibility fosters trust and engagement, ensuring that machine learning outputs are not confined to technical teams but can inform decision-making across an organization.

Ultimately, the project achieves its aim of testing stacked surrogate models and characterizing the results of different ensemble learning strategies, including both linear regression models (LRM) and large language models (LLM) adapted for surrogate tasks. In doing so, it contributes to the broader discourse on explainable AI, demonstrating that predictive power and transparency can coexist. The *Weather Aggregator Flask App* is not merely a forecasting tool; it is a narrative asset that illustrates best practices in ensemble modeling, surrogate characterization, and consulting-oriented system design. As such, it provides a foundation for future work in applying explainability-first ensemble learning to other domains, reinforcing the principle that innovation must be accompanied by accountability and clarity.

7. User Manual

Introduction

The Weather Aggregator is a modular, explainability-first forecasting system. It combines classical models (SARIMAX, Kalman), machine learning (Random Forest, Gradient Boosting), deep learning stubs (CNN, LSTM, Autoencoder), and hosted Hugging Face inference into a transparent, auditable ensemble.

This manual guides you through installation, usage, reporting, and governance features with practical examples.

Installation

1. Clone the repository

```
git clone https://github.com/your-org/weather-aggregator.git
cd weather-aggregator
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Verify installation

```
pytest tests/
```

You should see all unit tests passing.

Running the App

Option A: Terminal

```
python run.py
```

Visit `http://127.0.0.1:5000`.

Option B: Jupyter Notebook (interactive)

Paste this cell at the end of `01_flask_stub.ipynb`:

```
from threading import Thread
from run import create_app

app = create_app()

def run_flask():
    app.run(port=5000, debug=True, use_reloader=False)

thread = Thread(target=run_flask)
thread.start()

print("✅ Weather Aggregator app is running at http://127.0.0.1:5000")
```

User Interface

1. Home Page (/)

- Input form for **PO Box, City, Country**.
- Example:

```
PO Box: 12345  
City: Frankfurt  
Country: Germany
```

- Click **Submit** → Redirects to forecast dashboard.

2. Forecast Page (/forecast/<city>/<country>)

- Displays:
 - Current weather card (from OpenWeatherMap).
 - Forecasts from SARIMAX, Kalman, Random Forest, Hugging Face models.
 - Confidence bands (lower/upper bounds).
 - Ensemble forecast (meta learner).

Example Output:

```
Forecast for Frankfurt, Germany  
SARIMAX: [22.5, 23.0, 21.8]  
Kalman: [22.3, 22.4, 22.5]  
Random Forest: [22.0, 22.7, 23.1]  
Ensemble (Ridge): [22.3, 22.7, 22.5]  
Confidence Band: (21.0, 24.0)
```

3. Report Page (/report/<city>/<country>)

- Generates a **PDF report** with:
 - Forecast comparison chart.
 - Confidence bands.
 - Explainability summary (SHAP/LIME).
 - Metadata (model versions, run timestamp).

Step-by-Step Guide

1. Create a Hugging Face account

- Go to huggingface.co.
- Click **Sign Up** (top right).
- Register using your email, GitHub, or Google account.
- Verify your email address (this step is required before tokens can be created).

2. Navigate to Access Tokens

- Once logged in, click your **profile picture** (top right).
- Select **Settings**.
- In the left sidebar, choose **Access Tokens**.

3. Generate a new token

- Click **New Token**.
- Give it a descriptive name (e.g., `weather_app`).
- Choose a **role/scope**:
 - **Read** → allows downloading models/datasets (most common for inference).
 - **Write** → allows uploading models/datasets.
 - **Admin** → full access (use cautiously).
- Click **Generate**.

4. Copy and store the token securely

- The token will look like:

```
hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```



- Copy it and store it in a safe place.
- You'll use it in your app like:

```
import os
from huggingface_hub import InferenceApi

HF_TOKEN = os.getenv("HF_TOKEN") # or paste directly
api = InferenceApi(repo_id="bert-base-uncased", token=HF_TOKEN)
```



5. Best practices

- Do not **hardcode** the token in public repos.
- Use environment variables (`HF_TOKEN`) or a `.env` file.
- Rotate tokens periodically for security.



Sources

You can find detailed instructions on Hugging Face's official documentation: [User access tokens](#) and a step-by-step guide on [GeeksforGeeks](#).

✅ With this manual, we can install, run, and demonstrate the Weather Aggregator app in both technical and executive contexts.

8. References

1. **Chip Huyen**, AI Engineering: Building Applications with Foundation Models, 1st Edition, O'Reilly Media, 2025; **Michael Lanham**, AI Agents in Action, 1st Edition, Manning Publications, 2025; **Melanie Mitchell**, Artificial Intelligence: A Guide for Thinking Humans, 1st Edition, Pelican Books, 2019; **Brian Christian & Tom Griffiths**, Algorithms to Live By: The Computer Science of Human Decisions, 1st Edition, Henry Holt and Company, 2016; **Ray Kurzweil**, The Singularity Is Nearer: When We Merge with AI, 1st Edition, Viking, 2024; OpenWeatherMap: <https://openweathermap.org/>, HuggingFace: <https://huggingface.co/>, LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).
14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).

16. Volker Ziemann: "Physics and Finance", Springer (2021).
17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Annual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. Lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).
35. Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>. R. Nystrom: „Game Programming Patterns“, 1st Ed. Genevieve Benning (2014); A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);

36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);
37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).
38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1st Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024),

<https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>

39. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G. B. Dantzig, M. N. Thapa: "**Linear Programming 1: Introduction**", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: <https://www.cvxpy.org/>, <https://pypi.org/project/cvxpy/>; py-packages for operations research: <https://wiki.python.org/moin/PythonForOperationsResearch>
40. (Py-)tesseract package: <https://github.com/tesseract-ocr/tesseract>, <https://pypi.org/project/pytesseract/>, <https://builtin.com/data-science/python-ocr>, <https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/> and UB Mannheim builds.